

lab8_word2vec.py

Spiegazione dettagliata, riga per riga, del codice del Laboratorio 8

"Word Clustering Based on Word2Vec Representation"

Corso di Tecnologie del Linguaggio Naturale — Prof. Daniele Radicioni, UniTO

File analizzato: TLN/Parte2Radicioni/lab8_word2vec.py (603 righe)

Obiettivo del laboratorio: confrontare, tramite embedding Word2Vec addestrati separatamente, il linguaggio dei testi musicali degli anni '70 (Black Sabbath, Pink Floyd, Queen, Eagles...) con quello degli anni 2000 (Eminem, Coldplay, Taylor Swift, Linkin Park...), analizzando similarità semantiche, deriva del significato delle parole nel tempo, e clustering semantico via KMeans + t-SNE.

Come leggere questo documento: ogni sezione riprende un blocco del codice (nell'ordine in cui appare nel file) e lo espone in una tabella a tre colonne: numero di riga, codice sorgente, spiegazione. Le spiegazioni non descrivono solo *cosa* fa la riga, ma soprattutto **perché** è stata scritta così. Dove utile, un riquadro verde **"Esempio"** mostra una traccia passo-passo con valori **reali**, ottenuti eseguendo davvero lo script sul dataset (spotify_millsongdata.csv) e, dove necessario, con piccoli script di supporto per catturare dettagli interni non stampati dallo script originale.

0. Intestazione, import e setup (righe 1–40)

Import delle librerie, fix per l'encoding della console Windows, e preparazione della lista di stopwords (quella standard di NLTK più le contrazioni informali tipiche dei testi di canzoni).

Riga	Codice	Spiegazione
1–3	<pre># LAB 8 - Word Clustering Based on Word2Vec Representation # TLN - Daniele Radicioni, UNITO # Confronto: testi musicali anni 70 vs anni 2000</pre>	Commenti descrittivi: titolo del laboratorio e obiettivo — confrontare, tramite Word2Vec, il vocabolario e le relazioni semantiche di due epoche musicali diverse.
5–13	<pre>import re import sys import io import warnings import numpy as np import pandas as pd import matplotlib matplotlib.use('Agg') import matplotlib.pyplot as plt</pre>	<code>re</code> per le regex di pulizia testo; <code>sys / io</code> per il fix dell'encoding (riga 16); <code>warnings</code> per silenziare gli avvisi non critici di gensim; <code>numpy</code> per l'algebra vettoriale (embedding, matrici di similarità); <code>pandas</code> per caricare e filtrare il dataset CSV delle canzoni. <code>matplotlib.use('Agg')</code> va chiamato <i>prima</i> di importare <code>pyplot</code> per lo stesso motivo del lab4: forza un backend non interattivo che scrive su file, necessario perché lo script gira senza display grafico.
15–16	<pre># su Windows la console non e' UTF- 8 di default: senza questo i print con simboli non-ASCII crashano sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')</pre>	Perché: stessa problematica del lab4 (dove si usava <code>sys.stdout.reconfigure(encoding='utf-8')</code>), ma qui risolta ricostruendo esplicitamente lo stream con <code>io.TextIOWrapper</code> attorno al buffer grezzo — un modo alternativo e più portabile di ottenere lo stesso effetto (utile su versioni Python dove <code>.reconfigure()</code> non è disponibile). Senza questa riga, i caratteri come Δ o gli apostrofi tipografici nei print più avanti solleverebbero <code>UnicodeEncodeError</code> sulla codepage legacy di Windows.
18–23	<pre>import nltk from collections import Counter from nltk.corpus import stopwords from gensim.models import Word2Vec from sklearn.cluster import KMeans from sklearn.manifold import TSNE</pre>	Le librerie chiave dell'esperimento: <code>Word2Vec</code> (gensim) per addestrare gli embedding delle parole; <code>KMeans</code> (scikit-learn) per raggruppare le parole in cluster semantici sugli embedding; <code>TSNE</code> (scikit-learn) per proiettare gli embedding a 100 dimensioni in un piano 2D visualizzabile. <code>Counter</code> per le frequenze lessicali, come nel lab4.
25–26	<pre># silenzia i warning non critici di gensim, non i nostri warnings.filterwarnings('ignore', module=r'gensim.*')</pre>	Perché un filtro mirato al modulo <code>gensim</code>: gensim emette avvisi (es. su parametri deprecati o ottimizzazioni mancanti) che non riguardano la correttezza dell'esperimento. Il filtro usa una regex sul nome del modulo (<code>module=r'gensim.*'</code>) invece di <code>filterwarnings('ignore')</code> generico, così eventuali warning genuini sollevati dal <i>nostro</i> codice (es. da <code>pandas</code> o <code>numpy</code>) restano visibili — una scelta più prudente rispetto a silenziare tutto.

29-33	<pre>try: stop = set(stopwords.words('english')) except LookupError: nltk.download('stopwords') stop = set(stopwords.words('english'))</pre>	Perché un try/except invece di scaricare sempre (come faceva <code>nltk.download()</code> incondizionato nel lab4): qui si tenta prima di usare la risorsa già scaricata, ed effettua il download solo se manca (<code>LookupError</code>). È un pattern più efficiente quando lo script viene eseguito ripetutamente: evita la chiamata di rete/verifica di <code>nltk.download()</code> a ogni esecuzione se i dati sono già presenti in cache.
35-40	<pre># le contrazioni perdono l'apostrofo nella pulizia (don't -> dont), quindi # non sono coperte dalle stopwords NLTK: le aggiungiamo esplicitamente stop.update(['dont', 'cant', 'wont', 'aint', 'youre', 'youll', 'youve', 'ive', 'ill', 'im', 'id', 'hes', 'shes', 'thats', 'were', 'weve', 'theyre', 'lets', 'didnt', 'doesnt', 'isnt', 'wasnt', 'couldnt', 'wouldnt', 'theres'])</pre>	Motivazione precisa: la funzione <code>pulisci_testo</code> (righe 88-97) rimuove <i>tutti</i> i caratteri non alfabetici, apostrofo incluso, quindi <code>"don't"</code> diventa <code>"dont"</code> <i>prima</i> di essere confrontato con la lista di stopwords. La lista NLTK standard contiene però <code>"don't"</code> (con apostrofo), non <code>"dont"</code>: senza questo aggiornamento esplicito, tutte le contrazioni sopravvivrebbero al filtro come falsi "contenuti lessicali", inquinando le frequenze e (più avanti) gli embedding con token che sono in realtà solo forme verbali ausiliarie prive di significato tematico.
42-47	<pre>print("=" * 60) print("LAB 8 - Word2Vec su testi musicali") print("Anni 70 vs Anni 2000") print("=" * 60) print("\nCaricamento dataset...")</pre>	Banner di intestazione e primo messaggio di avanzamento — utile in uno script lungo (l'addestramento Word2Vec e il t-SNE richiedono tempo) per capire a che punto è arrivata l'esecuzione osservando l'output in tempo reale.

1. Caricamento dataset e selezione artisti per era (righe 49–84)

Il dataset è un CSV di 57.650 canzoni con testo completo (Kaggle "Spotify Million Song Dataset"). Da qui si estraggono due sottoinsiemi corrispondenti a due liste di artisti scelti manualmente come rappresentativi delle due epoche.

Riga	Codice	Spiegazione
49–50	<pre>df = pd.read_csv('spotify_millsongdata.csv') print(f"Dataset: {df.shape[0]} canzoni, {df['artist'].unique()} artisti")</pre>	Carica l'intero CSV in un DataFrame pandas. <code>df.shape[0]</code> è il numero di righe (canzoni); <code>.unique()</code> conta i valori distinti nella colonna <code>artist</code>. Output reale: Dataset: 57650 canzoni, 643 artisti — il dataset copre un intervallo molto più ampio di quanto servirà: solo 23 dei 643 artisti (12+11, righe 54–66) verranno effettivamente usati nell'analisi.
54–59	<pre>artisti_70s = ['Black Sabbath', 'Pink Floyd', 'Queen', 'Eagles', 'Fleetwood Mac', 'Bee Gees', 'David Bowie', 'Elton John', 'ABBA', 'Doors', 'Rolling Stones', 'Deep Purple']</pre>	Lista di 12 artisti scelti manualmente come rappresentativi del rock/pop anni '70 — una selezione qualitativa dell'autore dello script, non un criterio automatico (es. non filtrata per data di pubblicazione, dato che il CSV non contiene una colonna anno).
61–66	<pre>artisti_2000s = ['Eminem', 'Coldplay', 'Linkin Park', 'Christina Aguilera', 'Radiohead', 'Foo Fighters', 'Lady Gaga', 'Taylor Swift', 'The Killers', 'Green Day', 'Nickelback']</pre>	Lista simmetrica di 11 artisti rappresentativi degli anni 2000. Nota: gli artisti non sono perfettamente equivalenti in numero (12 vs 11) né in genere musicale — è una scelta pragmatica del laboratorio, non un campionamento statisticamente bilanciato; questo va tenuto presente nell'interpretare le differenze trovate più avanti (potrebbero riflettere differenze fra <i>questi specifici artisti</i> più che fra "le due epoche" in senso generale).
68–69	<pre>df_70s = df[df['artist'].isin(artisti_70s)]. copy() df_2000s = df[df['artist'].isin(artisti_2000s)]. copy()</pre>	<code>.isin()</code> filtra le righe il cui <code>artist</code> compare nella lista. <code>.copy()</code> è necessario perché pandas altrimenti restituirebbe una view sul DataFrame originale: scrivere successivamente su <code>df_70s['tokens']</code> (riga 99) senza <code>.copy()</code> solleverebbe il famoso <code>SettingWithCopyWarning</code> (o peggio, modificherebbe silenziosamente <code>df</code>).
71–74	<pre>print(f"Artisti 70s trovati: {sorted(df_70s['artist'].unique())} ")</pre>	Stampa di controllo con gli artisti effettivamente trovati (ordinati alfabeticamente per leggibilità) e il conteggio canzoni per era.

	<pre> print(f"Canzoni anni 70: {len(df_70s)}") print(f"\nArtisti 2000s trovati: {sorted(df_2000s['artist'].unique())})") print(f"Canzoni anni 2000: {len(df_2000s)}") </pre>	Output reale: 1729 canzoni per gli anni 70, 1312 per gli anni 2000 — un dataset sbilanciato di circa il 32% in più a favore degli anni 70, coerente con il fatto che la lista ha un artista in più (12 vs 11) ma anche con cataloghi di canzoni di dimensione diversa per artista.
76–84	<pre> # isin() non solleva errori se un nome non trova corrispondenze: verifica esplicita n_trovati_70s = df_70s['artist'].nunique() n_trovati_2000s = df_2000s['artist'].nunique() if n_trovati_70s != len(artisti_70s): mancanti = set(artisti_70s) - set(df_70s['artist'].unique()) print(f"ATTENZIONE: attesi {len(artisti_70s)} artisti anni 70, trovati {n_trovati_70s}. Mancanti: {mancanti}") if n_trovati_2000s != len(artisti_2000s): mancanti = set(artisti_2000s) - set(df_2000s['artist'].unique()) print(f"ATTENZIONE: attesi {len(artisti_2000s)} artisti anni 2000, trovati {n_trovati_2000s}. Mancanti: {mancanti}") </pre>	Motivazione esplicita nel commento: <code>.isin()</code> è "silenziosamente tollerante" — se in <code>artisti_70s</code> ci fosse un refuso di battitura (es. "Pink Flyod"), quell'artista verrebbe semplicemente ignorato senza errori né avvisi, e lo script proseguirebbe con meno dati del previsto senza che nessuno se ne accorga. Questo blocco calcola quanti nomi <i>distinti</i> sono stati effettivamente trovati e li confronta col numero atteso: se non coincidono, calcola la differenza fra insiemi (<code>set(...)</code> - <code>set(...)</code>) per stampare esattamente <i>quali</i> nomi non hanno trovato corrispondenza nel dataset. Nell'esecuzione reale nessun "ATTENZIONE" viene stampato: tutti e 12 gli artisti anni 70 e tutti e 11 gli artisti anni 2000 sono stati trovati esattamente (confrontare l'elenco della riga 71 con le liste delle righe 54–66: coincidono, solo riordinati alfabeticamente). Questo blocco è quindi una rete di sicurezza silenziosa che qui non scatta, ma che avrebbe individuato subito un eventuale errore di battitura nei nomi degli artisti.

2. Pulizia dei testi (righe 86–113)

Ogni testo di canzone (colonna `text` del CSV) viene tokenizzato e ripulito da annotazioni strutturali, punteggiatura, stopwords e parole troppo corte, per ottenere le sequenze di token su cui addestrare Word2Vec.

Riga	Codice	Spiegazione
88–91	<pre>def pulisci_testo(testo): """Tokenizza un testo di canzone rimuovendo annotazioni, punteggiatura, stopwords e parole corte.""" if not isinstance(testo, str): return []</pre>	Perché il controllo <code>isinstance</code> : in un CSV letto con pandas, un campo testo mancante viene caricato come <code>NaN</code> (un <code>float</code> , non una stringa). Senza questo controllo, <code>re.sub(..., testo)</code> alla riga successiva solleverebbe <code>TypeError</code> su qualunque canzone con testo mancante. Restituire <code>[]</code> (lista vuota) invece di sollevare un errore permette allo script di continuare, e la canzone verrà comunque scartata più avanti dal filtro <code>tokens.map(len) > 5</code> (riga 102).
92	<pre>testo = re.sub(r'\[.*?\]', '', testo)</pre>	Rimuove le annotazioni strutturali tra parentesi quadre tipiche dei testi di canzoni (es. <code>[Chorus]</code> , <code>[Verse 2]</code> , <code>[Bridge]</code>). Il quantificatore <i>non goloso</i> <code>.*</code> ? è essenziale: con un quantificatore goloso <code>.*</code> e più annotazioni nello stesso testo, il match si estenderebbe dalla prima <code>[</code> all' <i>ultima</i> <code>]</code> dell'intero testo, cancellando anche tutte le parole di testo vero in mezzo.
93–94	<pre>testo = re.sub(r'^[a-zA-Z\s]', '', testo) testo = testo.lower()</pre>	Stessa strategia del lab4: elimina tutto ciò che non è lettera o spazio (punteggiatura, numeri, apostrofi, ritorni a capo <code>\r\n</code>) e minuscolizza. Effetto collaterale importante : eliminando <code>\r\n</code> insieme alla punteggiatura, le interruzioni di riga del testo della canzone spariscono silenziosamente, lasciando solo gli spazi già presenti — è per questo che alla riga successiva basta <code>.split()</code> semplice (che spezza su <i>qualunque</i> sequenza di spazi bianchi) invece di un tokenizer dedicato come <code>word_tokenize</code> usato nel lab4.
95–97	<pre>tokens = testo.split() tokens = [t for t in tokens if t not in stop and len(t) >= 3] return tokens</pre>	<code>.split()</code> senza argomenti spezza su spazi/tab/newline multipli e scarta le stringhe vuote risultanti — più semplice ed economico di <code>word_tokenize</code> perché qui non serve gestire punteggiatura (già rimossa alla riga 93). Il filtro finale scarta le stopwords e i token di meno di 3 caratteri: perché anche la soglia di lunghezza — dopo aver spezzato parole come "I'm", "it's" in "im", "its" (2-3 carattere), molti frammenti brevi residui sono rumore (es. "uh", "la", inflessioni vocali tipiche dei testi musicali) piuttosto che parole di contenuto; la soglia <code>len(t) >= 3</code> è un filtro aggiuntivo euristico, oltre alla lista di stopwords esplicite. Traccia reale su una canzone vera del dataset (ABBA – "Ahe's My Kind Of Girl", prima canzone di <code>df_70s</code>):

Fase	Contenuto (primi ~200 caratteri o token)
testo grezzo	"Look at her face, it's a wonderful face \r\nAnd it means something special to me \r\nLook at the way that she smiles when she sees me \r\n..."
dopo riga 92 (annotazioni)	invariato – questa canzone non contiene [...]
dopo riga 93 (non- alfa)	"Look at her face its a wonderful face \r\nAnd it means something special to me \r\n..." <i>(virgola e apostrofo spariti, "it's"→"its")</i>
dopo riga 94 (lower)	"look at her face its a wonderful face \r\nand it means something special to me \r\n..."
dopo riga 95 (.split())	['look','at','her','fa ce','its','a','wonderf ul','face','and','it', 'means','something','s pecial','to','me','loo k','at','the','way','t hat', ...] – 153 token totali
dopo riga 96 (filtro)	['look','face','wonder ful','face','means','s omething','special','l ook','way','smiles','s ees','lucky','one','fe llow','kind','girl','m akes','feel','fine','c ould', ...] – 61 token superstiti

Da 153 token grezzi (dopo lo split) ne sopravvivono 61: quasi il 60% del testo — parole come "at", "her", "its", "a", "and", "it", "to", "me", "the", "that" — viene scartato in quanto stopword o troppo corto. Da notare come i \r\n (ritorni a capo tra i versi) siano semplicemente scomparsi nel testo pulito: .split() li ha trattati come normali separatori di token, quindi l'informazione su dove finisce un verso e ne comincia un altro è persa —

		Word2Vec vedrà l'intera canzone come un'unica sequenza continua di parole.
99-100	<pre>df_70s['tokens'] = df_70s['text'].apply(pulisci_testo) df_2000s['tokens'] = df_2000s['text'].apply(pulisci_testo)</pre>	<code>.apply()</code> chiama <code>pulisci_testo</code> su ogni valore della colonna <code>text</code>, creando una nuova colonna <code>tokens</code> con la lista di token risultante per ciascuna canzone.
102-103	<pre>df_70s = df_70s[df_70s['tokens'].map(len) > 5] df_2000s = df_2000s[df_2000s['tokens'].map(len) > 5]</pre>	Scarta le canzoni con 5 o meno token superstiti dopo la pulizia (es. testi strumentali, testi mancanti diventati <code>[]</code> alla riga 91, o canzoni quasi interamente fatte di stopword). Perché una soglia così bassa (5, non es. 20 o 50): l'obiettivo qui è solo scartare i casi patologici (lista vuota o quasi), non filtrare per "qualità" del testo — canzoni brevi ma genuine devono comunque contribuire al training di Word2Vec. Nell'esecuzione reale, per gli anni 70 questo filtro non scarta nessuna canzone: 1729 prima, 1729 dopo — segno che tutte le canzoni della lista di 12 artisti hanno un testo sufficientemente ricco già dopo la pulizia.
105-106	<pre>tutti_70s = [t for tok in df_70s['tokens'] for t in tok] tutti_2000s = [t for tok in df_2000s['tokens'] for t in tok]</pre>	List comprehension annidata che "appiattisce" la colonna <code>tokens</code> (una lista di liste, una per canzone) in un'unica lista piatta di tutti i token di tutte le canzoni dell'era — analoga a <code>testo_tweet_unito</code> del lab4, qui usata solo per le statistiche di frequenza (righe 115 e seguenti), non per l'addestramento (Word2Vec riceverà le liste <i>separate</i> per canzone, riga 141, per lo stesso motivo dei confini di sequenza visto nel lab4).
108-113	<pre>print(f"Anni 70 → {len(df_70s)} canzoni, " f"{len(tutti_70s)} token, " f"{len(set(tutti_70s))} unici") print(f"Anni 2000 → {len(df_2000s)} canzoni, " f"{len(tutti_2000s)} token, " f"{len(set(tutti_2000s))} unici")</pre>	Stampa statistiche dimensionali dei due corpora dopo la pulizia. Output reale: anni 70 → 1729 canzoni, 163.180 token, 11.174 parole uniche; anni 2000 → 1312 canzoni, 149.820 token, 11.551 parole uniche. Interessante: pur avendo <i>meno</i> canzoni e <i>meno</i> token totali, gli anni 2000 hanno un vocabolario <i>leggermente più ampio</i> (11.551 vs 11.174 parole uniche) — un primo indizio di maggiore varietà lessicale per canzone, coerente con generi come il rap (es. Eminem) che tendono a un vocabolario più denso.

3. Frequenze lessicali (righe 115–124)

Conteggio delle parole più frequenti in ciascuna era, come primo confronto (grossolano) prima di passare agli embedding veri e propri.

Riga	Codice	Spiegazione
115–116	<pre>freq_70s = Counter(tutti_70s) freq_2000s = Counter(tutti_2000s)</pre>	Costruisce le tabelle di frequenza assoluta di ogni parola in ciascuna era. Questi due <code>Counter</code> verranno riutilizzati per tutto il resto dello script (es. per filtrare le parole "abbastanza frequenti" nelle sezioni 8 e 9), non solo per questa stampa.
118–124	<pre>print("\nTop 15 parole anni 70:") for p, c in freq_70s.most_common(15): print(f" {p:<15} → {c}") print("\nTop 15 parole anni 2000:") for p, c in freq_2000s.most_common(15): print(f" {p:<15} → {c}")</pre>	Stampa le 15 parole più frequenti per era. Output reale (prime 5 di ciascuna): anni 70 → love (2669), know (1635), got (1500), like (1458), baby (1374); anni 2000 → like (1847), know (1780), love (1334), get (1244), one (1240). Le liste sono molto simili (parole molto generiche dominano in entrambe le epoche: "love", "know", "like", "got/get"), a conferma che le semplici frequenze assolute dicono poco sulle differenze semantiche fra le due ere — è proprio questa la motivazione per passare a Word2Vec: due parole possono avere frequenza simile ma un contesto d'uso (i "vicini" nello spazio degli embedding) completamente diverso, come si vedrà nella sezione 9.

4. Addestramento Word2Vec (righe 126–145)

Vengono addestrati due modelli Word2Vec completamente separati, uno per era, sulle rispettive liste di canzoni tokenizzate. Ogni modello impara una rappresentazione vettoriale (embedding) per ciascuna parola, basata su quali parole tendono a comparire vicino ad essa nel corpus di training.

Riga	Codice	Spiegazione
128–139	<pre>params = dict(sg = 1, # Skip- gram (il default di gensim sarebbe CBOW, sg=0) vector_size = 100, window = 5, min_count = 3, workers = 1, # riproducibilita' bit-esatta: con workers>1 l'ordine di # elaborazione tra thread varia e il training smette di essere # deterministico anche a parita' di seed (costo trascurabile # su un corpus di questa dimensione) epochs = 20, seed = 42)</pre>	Dizionario di iperparametri condiviso da entrambi i modelli (passato poi con <code>**params</code>), scelta che garantisce che le uniche differenze fra <code>model_70s</code> e <code>model_2000s</code> siano nei dati di training, non negli iperparametri. • <code>sg=1</code> (skip-gram): il modello impara a predire le parole di <i>contesto</i> a partire da una parola <i>centrale</i> data (l'opposto di CBOW, che predice la parola centrale dato il contesto). Perché skip-gram e non il default CBOW: skip-gram è generalmente più efficace con corpora piccoli e con parole rare (qui il corpus è di sole ~150k parole per era, molto piccolo per gli standard di Word2Vec, che tipicamente lavora su corpora di miliardi di parole). • <code>vector_size=100</code>: ogni parola viene rappresentata da un vettore di 100 numeri reali — un compromesso fra espressività (vettori più lunghi catturano più sfumature) e overfitting/tempo di training (con un corpus piccolo, vettori troppo lunghi rischiano di non avere abbastanza dati per stimare bene ogni dimensione). • <code>window=5</code>: per ogni parola centrale, il contesto considerato è le 5 parole prima e le 5 dopo (dopo la rimozione delle stopword, quindi in realtà una finestra semanticamente più "ampia" di 5 parole di superficie, dato che le parole funzionali intermedie sono già state tolte). • <code>min_count=3</code>: le parole che compaiono meno di 3 volte nell'intero corpus di quell'era vengono escluse dal vocabolario del modello — non c'è abbastanza segnale statistico per stimarne un embedding affidabile. • <code>workers=1</code>: come spiegato nel commento del codice, il training multi-thread di gensim introduce non-determinismo nell'ordine in cui i thread aggiornano i pesi condivisi, quindi anche fissando <code>seed=42</code> il risultato varierebbe leggermente fra esecuzioni diverse. Con <code>workers=1</code> (thread singolo) il training è completamente riproducibile bit-esatto, al costo di essere più lento — costo giudicato accettabile qui vista la dimensione ridotta del corpus. • <code>epochs=20</code>: il corpus viene attraversato 20 volte durante il training (più del default di gensim, che è 5) — necessario perché un corpus di sole ~150k parole è piccolo, quindi servono più passate per convergere a embedding stabili.

		• <code>seed=42</code> : fissa il generatore di numeri casuali usato per l'inizializzazione dei vettori e per il negative sampling, rendendo l'intero esperimento riproducibile (a parità di <code>workers=1</code>).
141-142	<pre>model_70s = Word2Vec(sentences=df_70s['tokens'] .tolist(), **params) model_2000s = Word2Vec(sentences=df_2000s['tokens'] .tolist(), **params)</pre>	<code>sentences</code> vuole una lista di liste di token (una lista per "frase" — qui, una lista per canzone, tramite <code>.tolist()</code> sulla colonna <code>tokens</code> del DataFrame). Perché canzoni separate e non un unico flusso: stesso principio del lab4 — Word2Vec, come i modelli n-gram, apprende dalle co-occorrenze <i>all'interno</i> di ciascuna sequenza (qui, all'interno della finestra di contesto di 5 parole); tenere le canzoni separate evita che le ultime parole di una canzone vengano trattate come "vicine" alle prime parole della canzone successiva. <code>**params</code> espande il dizionario di iperparametri in argomenti keyword individuali.
144-145	<pre>print(f"Vocabolario anni 70: {len(model_70s.wv)} parole") print(f"Vocabolario anni 2000: {len(model_2000s.wv)} parole")</pre>	<code>model.wv</code> è l'oggetto "word vectors" del modello addestrato; <code>len()</code> restituisce quante parole hanno superato la soglia <code>min_count=3</code> ed entrano quindi nel vocabolario con un proprio embedding. Output reale: vocabolario anni 70 = 4752 parole, anni 2000 = 4800 parole — molto più piccolo delle 11.174/11.551 parole "uniche" contate alla sezione 2: la differenza ($11.174 - 4752 \approx 6400$ parole) sono tutte le parole che compaiono 1 o 2 volte nel corpus e vengono quindi escluse da <code>min_count=3</code> perché troppo rare per un embedding affidabile.

5. Parole simili per parola chiave (righe 147–159)

Primo uso concreto degli embedding: per alcune parole chiave, si chiede a ciascun modello quali sono le parole più vicine nello spazio semantico appreso.

Riga	Codice	Spiegazione																					
149	<pre>parole_test = ['love', 'night', 'heart', 'world', 'time', 'life', 'feel']</pre>	Sette parole scelte come "sonde" semantiche — termini generici, presumibilmente comuni e presenti in entrambi i vocabolari, adatti a un confronto diretto fra le due epoche.																					
151–152	<pre>print(f"\n{'PAROLA':<10} {'TOP 3 ANNI 70':<35} TOP 3 ANNI 2000") print("-" * 80)</pre>	Intestazione della tabella testuale a colonne fisse.																					
154–159	<pre>for parola in parole_test: sim_70 = [w for w,_ in model_70s.wv.most_similar(parola, topn=3)] \ if parola in model_70s.wv else ['---'] sim_2000 = [w for w,_ in model_2000s.wv.most_similar(parola, topn=3)] \ if parola in model_2000s.wv else ['---'] print(f"{'parola':<10} {'', '.join(sim_70):<35} {'', '.join(sim_2000)}")</pre>	<code>most_similar(parola, topn=3)</code> restituisce le 3 parole con coseno-similarità più alta rispetto a <code>parola</code>, come lista di coppie (<code>parola</code>, <code>punteggio</code>) — la list comprehension <code>[w for w,_ in ...]</code> tiene solo le parole, scartando i punteggi numerici. Perché il controllo <code>if parola in model.wv</code>: se la parola chiave non fosse nel vocabolario del modello (perché troppo rara, esclusa da <code>min_count=3</code>), <code>most_similar()</code> solleverebbe <code>KeyError</code> e lo script si interromperebbe; il fallback <code>['---']</code> permette di continuare mostrando un placeholder. Output reale (tutte e 7 le parole erano presenti in entrambi i vocabolari): <table><tr><th>parola</th><th>top 3 anni 70</th><th>top 3 anni 2000</th></tr><tr><td>love</td><td>forsake, harmony, tender</td><td>everlastic, lusting, elastic</td></tr><tr><td>night</td><td>tonights, starry, gloria</td><td>mmmarry, friday, mmmmarry</td></tr><tr><td>heart</td><td>hijack, indecision, steals</td><td>aaate, amaze, belongs</td></tr><tr><td>world</td><td>battleground, smash, upside</td><td>slips, rule, seas</td></tr><tr><td>time</td><td>rages, rediscover, enemies</td><td>thanks, wasting, replaced</td></tr><tr><td>life</td><td>lalala, earn, lalena</td><td>intoxicated, overdrive,</td></tr></table>	parola	top 3 anni 70	top 3 anni 2000	love	forsake, harmony, tender	everlastic, lusting, elastic	night	tonights, starry, gloria	mmmarry, friday, mmmmarry	heart	hijack, indecision, steals	aaate, amaze, belongs	world	battleground, smash, upside	slips, rule, seas	time	rages, rediscover, enemies	thanks, wasting, replaced	life	lalala, earn, lalena	intoxicated, overdrive,
parola	top 3 anni 70	top 3 anni 2000																					
love	forsake, harmony, tender	everlastic, lusting, elastic																					
night	tonights, starry, gloria	mmmarry, friday, mmmmarry																					
heart	hijack, indecision, steals	aaate, amaze, belongs																					
world	battleground, smash, upside	slips, rule, seas																					
time	rages, rediscover, enemies	thanks, wasting, replaced																					
life	lalala, earn, lalena	intoxicated, overdrive,																					

		kandy
feel	sheer, cardiac, tense	caren, garner, alienated

Da notare (interpretazione critica, non solo

descrizione): molti dei "vicini" trovati sono parole rare o quasi-uniche (es. "gloria", "hijack", "lalena", "mmmarry", "kandy") più che sinonimi tematici intuitivi. Questo è un effetto atteso quando si addestra Word2Vec su un corpus piccolo (~150k token per era, mentre Word2Vec è tipicamente usato su corpora di miliardi di parole): le parole rare tendono ad "agganciarsi" ad altre parole altrettanto rare che sono comparse per coincidenza nello stesso ristretto contesto, producendo vicini semanticamente meno intuitivi di quanto ci si aspetterebbe da un modello addestrato su Wikipedia o Google News. Va tenuto a mente come limite metodologico nell'interpretare tutti i risultati successivi basati su `most_similar()`.

6. Similarità coseno tra coppie di parole (righe 161–180)

Invece di chiedere "quali sono le parole più vicine a X", qui si misura direttamente quanto sono vicine due parole specifiche, per coppie scelte per il loro potenziale interesse semantico o culturale.

Riga	Codice	Spiegazione																
163-170	<pre>coppie = [\n ('love', 'heart'),\n ('love', 'money'),\n ('night', 'dark'),\n ('god', 'heaven'),\n ('war', 'peace'),\n ('free', 'soul'),\n]</pre>	Sei coppie di parole scelte manualmente, ciascuna con un possibile significato interpretativo: es. "love vs money" per vedere se il romanticismo è più o meno associato al materialismo fra le due epoche, "war vs peace" per un tema più politico/sociale.																
172-173	<pre>print(f"\n{'Coppia':<22} {'Anni 70':>8} {'Anni 2000':>10}")\nprint("-" * 48)</pre>	Intestazione della tabella.																
175-180	<pre>for w1, w2 in coppie:\n s70 = f"\n{model_70s.wv.similarity(w1,w2):.4f}\n \n if w1 in model_70s.wv\nand w2 in model_70s.wv else "---"\n s2000 = f"\n{model_2000s.wv.similarity(w1,w2):.4f}\n \n if w1 in model_2000s.wv\nand w2 in model_2000s.wv else "---"\n print(f"({w1}, {w2}){'':6} {s70:>8} {s2000:>10}")</pre>	<code>.similarity(w1, w2)</code> restituisce direttamente il coseno fra i due vettori embedding (un numero fra -1 e 1, dove valori più alti indicano parole usate in contesti più simili). Il controllo di appartenenza al vocabolario è fatto per <i>entrambe</i> le parole della coppia (a differenza della sezione 5, qui serve <code>and</code> perché la similarità richiede che <i>entrambe</i> le parole abbiano un embedding). Output reale (tutte le coppie erano presenti in entrambi i vocabolari):<table><tr><th>coppia</th><th>anni 70</th><th>anni 2000</th><th>osservazione</th></tr><tr><td>(love, heart)</td><td>0.3260</td><td>0.2623</td><td>leggermente meno associate nei 2000</td></tr><tr><td>(love, money)</td><td>0.1579</td><td>0.3203</td><td>quasi raddoppiata: più associate nei 2000</td></tr><tr><td>(night, dark)</td><td>0.2860</td><td>0.3300</td><td>simile, leggero aumento</td></tr></table>	coppia	anni 70	anni 2000	osservazione	(love, heart)	0.3260	0.2623	leggermente meno associate nei 2000	(love, money)	0.1579	0.3203	quasi raddoppiata: più associate nei 2000	(night, dark)	0.2860	0.3300	simile, leggero aumento
coppia	anni 70	anni 2000	osservazione															
(love, heart)	0.3260	0.2623	leggermente meno associate nei 2000															
(love, money)	0.1579	0.3203	quasi raddoppiata: più associate nei 2000															
(night, dark)	0.2860	0.3300	simile, leggero aumento															

(god, heaven)	0.2809	0.2020	meno associate nei 2000
(war, peace)	0.3975	0.3767	simile in entrambe le ere
(free, soul)	0.2680	0.1335	dimezzata : molto meno associate nei 2000

Il caso più marcato è (love, money) , che raddoppia da 0.1579 a 0.3203: nei testi degli anni 2000 selezionati (che includono Eminem, un rapper il cui repertorio tocca spesso temi economici/materiali insieme a quelli romantici) le due parole tendono a comparire in contesti più simili rispetto agli artisti rock/pop anni 70 selezionati. È un'osservazione interessante ma da maneggiare con cautela: riflette gli *artisti scelti* (righe 54–66), non necessariamente "gli anni 2000" come fenomeno culturale generale.

7. Analisi del vocabolario (righe 182–207)

Confronto insiemistico dei due vocabolari: quali parole sono esclusive di un'epoca, quali sono condivise, e quali fra le parole esclusive sono abbastanza frequenti da essere significative (non semplici refusi o rarità).

Riga	Codice	Spiegazione
184–185	<pre>vocab_70s = set(model_70s.wv.index_to_key) vocab_2000s = set(model_2000s.wv.index_to_key)</pre>	<code>model.wv.index_to_key</code> è la lista ordinata (per frequenza decrescente) di tutte le parole nel vocabolario del modello; convertita in <code>set</code> per abilitare le operazioni insiemistiche della riga successiva.
187–189	<pre>solo_70s = vocab_70s - vocab_2000s solo_2000s = vocab_2000s - vocab_70s comuni = vocab_70s & vocab_2000s</pre>	Operazioni standard su insiemi: differenza (<code>-</code>) per le parole esclusive di un'epoca, intersezione (<code>&</code>) per le parole condivise. Questi tre insiemi verranno riutilizzati per il resto dello script (sezioni 8 e 9).
191–193	<pre>print(f"Solo anni 70: {len(solo_70s)} parole") print(f"Solo anni 2000: {len(solo_2000s)} parole") print(f"In comune: {len(comuni)} parole")</pre>	Output reale: solo anni 70 → 1779 parole; solo anni 2000 → 1827 parole; in comune → 2973 parole. Verifica di coerenza: $1779+2973 = 4752$ (= vocabolario anni 70, sezione 4) e $1827+2973 = 4800$ (= vocabolario anni 2000) — i numeri tornano esattamente, come deve essere per definizione di differenza/intersezione insiemistica.
195–202	<pre>interessanti_70s = sorted([p for p in solo_70s if freq_70s[p] > 20], key=lambda x: (-freq_70s[x], x)) interessanti_2000s = sorted([p for p in solo_2000s if freq_2000s[p] > 20], key=lambda x: (-freq_2000s[x], x))</pre>	Filtra, fra le parole <i>esclusive</i> di un'epoca, solo quelle abbastanza frequenti (più di 20 occorrenze) da essere un segnale genuino e non un refuso isolato o un nome proprio marginale. <code>key=lambda x: (-freq_70s[x], x)</code> : ordina per frequenza decrescente (il segno meno inverte l'ordine naturale crescente di <code>sorted</code>) e, a parità di frequenza, alfabeticamente (<code>x</code> come secondo criterio di spartizione) — la stessa tecnica di ordinamento a tupla vista nel lab4, che garantisce un ordine deterministico anche quando più parole condividono la stessa frequenza esatta.
204–207	<pre>print(f"\nParole caratteristiche anni 70 (freq > 20):") print(f" {interessanti_70s[:20]}") print(f"\nParole caratteristiche anni 2000 (freq > 20):") print(f" {interessanti_2000s[:20]}")</pre>	Stampa solo le prime 20 (le più frequenti, grazie all'ordinamento) di ciascuna lista. Output reale (prime 10 di ciascuna): Anni 70: boogie, diddley, whop, aaaahh, ahha, mustapha, gentle, zane, bicycle, chingaling... Anni 2000: superstar, hooker, gangsta, yall, rap, muh, retro, ayo, chocolate, freezer...

Le parole esclusive degli anni 2000 riflettono chiaramente l'influenza dell'hip-hop/rap nel sottoinsieme di artisti scelto (es. "gangsta", "yall", "rap", "ayo" sono forme gergali tipiche di quel genere, assenti nel rock/pop anni 70), mentre quelle degli anni 70 riflettono onomatopee ed espressioni vocali tipiche del rock dell'epoca ("boogie", "diddley" — riferimento a Bo Diddley — "whop", suoni tipo "aaaahh").

8. Analisi sistematica delle coppie di parole (righe 209–272)

Invece delle 6 coppie scelte a mano della sezione 6, qui si confrontano **tutte** le coppie possibili fra le parole abbastanza frequenti in entrambe le epoche, per trovare sistematicamente quali relazioni semantiche sono cambiate di più (e quali sono rimaste più stabili).

Riga	Codice	Spiegazione
211	<pre>FREQ_COPPIE = 100</pre>	Soglia di frequenza minima (in <i>entrambe</i> le epoche) per includere una parola nell'analisi sistematica. Perché 100 e non 20 come alla sezione 7: lì si cercavano parole caratteristiche anche rare purché non isolate; qui invece si vogliono solo parole così frequenti che il loro embedding sia stimato in modo affidabile (con poche occorrenze, l'embedding di una parola è rumoroso e le similarità calcolate su di esso sarebbero poco significative) — una soglia molto più stringente per un'analisi quantitativa su larga scala.
212–214	<pre># sorted(): l'intersezione fra due set ha un ordine di iterazione dipendente da # PYTHONHASHSEED (randomizzato ad ogni processo Python); senza un ordine fisso, # le liste con parità esatta di punteggio (vedi STEP 9) sarebbero non riproducibili</pre>	Stessa problematica del lab4 (dove <code>sorted()</code> serviva a rendere deterministico l'ordine di spareggio dopo un sort a parità di punteggio): qui <code>vocab_70s & vocab_2000s</code> è un <code>set</code> , il cui ordine di iterazione in Python dipende dall'hash delle stringhe, che è randomizzato a ogni avvio dell'interprete (per motivi di sicurezza, a partire da Python 3.3) a meno di fissare <code>PYTHONHASHSEED</code> . Passare l'output attraverso <code>sorted()</code> (ordine alfabetico) rende <code>candidati</code> — e quindi tutto ciò che ne dipende, incluso l'ordine di spareggio nel sort della riga 247 — identico a ogni esecuzione dello script.
215–220	<pre>candidati = [p for p in sorted(vocab_70s & vocab_2000s) if freq_70s[p] >= FREQ_COPPIE and freq_2000s[p] >= FREQ_COPPIE] print(f"Parole candidate (freq >= {FREQ_COPPIE} in entrambe): {len(candidati)}") print(f"Coppie totali da confrontare: {len(candidati) * (len(candidati)-1) // 2}")</pre>	Filtra le parole comuni ad entrambi i vocabolari, tenendo solo quelle con almeno 100 occorrenze in <i>entrambe</i> le epoche. <code>len(candidati) * (len(candidati)-1) // 2</code> è la formula per il numero di coppie non ordinate distinte da n elementi (combinazioni di n a 2: $n(n-1)/2$), usata solo per una stampa informativa <i>prima</i> di calcolarle davvero. Output reale: 245 parole candidate → $245 \times 244 / 2 = 29.890$ coppie — esattamente il numero stampato dallo script (verifica aritmetica: la formula è corretta). Primi 5 candidati in ordine alfabetico: alive, alone, alright, always, another .
222–227	<pre># similarita' vettorizzata: una singola matrice N x N invece di N^2/2 chiamate # a .similarity() (rilevante quando i candidati salgono a qualche centinaio)</pre>	Perché vettorizzare invece di chiamare <code>.similarity()</code> 29.890 volte: ogni chiamata a <code>.similarity()</code> ha un overhead Python non trascurabile; con centinaia di candidati (quindi decine di migliaia di coppie) conviene calcolare tutte le similarità in un colpo solo con algebra matriciale, molto più veloce perché eseguita in codice C vettorizzato di numpy. Il

```
def matrice_coseno(model, parole):
    vettori = np.array([model.wv[p]
for p in parole])
    norme = vettori /
np.linalg.norm(vettori, axis=1,
keepdims=True)
    return norme @ norme.T
```

procedimento: (1) `vettori` è una matrice $N \times 100$ (N parole, ciascuna con il suo embedding a 100 dimensioni); (2) `norme` divide ogni riga per la propria norma euclidea (`axis=1` = norma calcolata per riga, `keepdims=True` mantiene la forma a colonna per la divisione broadcast), ottenendo vettori di lunghezza 1; (3) `norme @ norme.T` è il prodotto matriciale fra la matrice normalizzata e la sua trasposta, che produce una matrice $N \times N$ dove l'elemento `[i,j]` è il prodotto scalare fra il vettore i -esimo e il j -esimo normalizzati — che è per definizione il coseno di similarità fra le due parole (coseno = prodotto scalare / prodotto delle norme; avendo già normalizzato a norma 1, il prodotto scalare grezzo coincide col coseno).

```
229-230 sim70 = matrice_coseno(model_70s,
candidati)
sim2000 =
matrice_coseno(model_2000s,
candidati)
```

Calcola le due matrici 245×245 di similarità coseno, una per era, usando lo stesso ordine di parole (`candidati`) per entrambe — essenziale perché più avanti si accede a `sim70[i,j]` e `sim2000[i,j]` assumendo che l'indice `i` si riferisca alla stessa parola in entrambe le matrici.

```
232-245 coppie_sist = []
for i, w1 in enumerate(candidati):
    for j in range(i + 1,
len(candidati)):
        w2 = candidati[j]
        s70 = float(sim70[i, j])
        s2000 = float(sim2000[i,
j])
        coppie_sist.append({
            'w1': w1,
            'w2': w2,
            's70': s70,
            's2000': s2000,
            'delta': abs(s70 -
s2000),
            'direzione': s2000 -
s70,
        })
```

Doppio ciclo che enumera tutte le coppie *non ordinate* distinte: il ciclo interno parte da `i+1` (non da 0), così ogni coppia (`w1,w2`) viene generata una sola volta e non si include mai (`w1,w1`) — è esattamente la logica che produce le 29.890 combinazioni contate alla riga 220. Per ogni coppia, estrae le due similarità dalle matrici precalcolate (`float(...)` converte da `numpy.float32` a `float` Python nativo, più comodo da formattare/confrontare) e costruisce un dizionario con entrambi i punteggi, il `delta` assoluto (quanto è cambiata la similarità, indipendentemente dalla direzione) e la `direzione` con segno (positiva = le parole si sono avvicinate nei 2000, negativa = si sono allontanate).

Prima iterazione reale del doppio ciclo (`i=0`,
`w1=candidati[0]="alive"`; `j=1`,
`w2=candidati[1]="alone"`):

Variabile	Valore
<code>w1, w2</code>	'alive', 'alone'
<code>s70 = sim70[0,1]</code>	0.1716
<code>s2000 = sim2000[0,1]</code>	0.2033
<code>delta = abs(s70-s2000)</code>	0.0317
<code>direzione = s2000-s70</code>	+0.0317 (positiva → più vicine nei 2000)

Il ciclo interno per `i=0` prosegue poi con `j=2,3,4,...,244` (confrontando "alive" con tutte le altre 243 parole successive), poi passa a `i=1` (`w1="alone"`) con `j` da 2 in su, e così via, fino a coprire tutte le 29.890 coppie.

247	<pre>coppie_sist.sort(key=lambda x: (- x['delta'], x['w1'], x['w2']))</pre>	Ordina la lista per <code>delta</code> decrescente (coppie più "cambiate" per prime), con spareggio alfabetico su <code>w1</code> poi <code>w2</code> per un ordine deterministico in caso di <code>delta</code> identici — di nuovo la stessa tecnica di ordinamento a tupla del lab4.																														
258-261	<pre>for r in coppie_sist[:20]: trend = "piu' vicine ->" if r['direzione'] > 0 else "<- piu' lontane" coppia = f"({r['w1']}, {r['w2']})" print(f"{coppia:<26} {r['s70']:>8.4f} {r['s2000']:>10.4f} {r['delta']:>7.4f} {trend}")</pre>	Stampa le 20 coppie con il maggior cambiamento di similarità, con un'etichetta testuale di direzione basata sul segno di <code>direzione</code>. Output reale (prime 5 su 20): <table><tr><th>coppia</th><th>anni 70</th><th>anni 2000</th><th>delta</th><th>trend</th></tr><tr><td>(tell, world)</td><td>0.4472</td><td>0.0002</td><td>0.4470</td><td>più lontane</td></tr><tr><td>(fight, two)</td><td>-0.0777</td><td>0.3315</td><td>0.4092</td><td>più vicine</td></tr><tr><td>(lies, remembe r)</td><td>0.4242</td><td>0.0162</td><td>0.4081</td><td>più lontane</td></tr><tr><td>(happy, show)</td><td>0.4278</td><td>0.0219</td><td>0.4059</td><td>più lontane</td></tr><tr><td>(city, old)</td><td>-0.0944</td><td>0.3048</td><td>0.3992</td><td>più vicine</td></tr></table> La coppia col cambiamento più marcato in assoluto è (tell, world) : da fortemente associate negli anni 70 (0.4472) a praticamente scorrelate nei 2000 (0.0002, quasi zero). Da notare anche che alcuni valori di similarità sono <i>negativi</i> (es. -0.0777, -0.0944): il coseno può variare fra -1 e 1, e un valore negativo indica che le due parole tendono a comparire in contesti quasi <i>opposti</i>, non semplicemente "non correlati".	coppia	anni 70	anni 2000	delta	trend	(tell, world)	0.4472	0.0002	0.4470	più lontane	(fight, two)	-0.0777	0.3315	0.4092	più vicine	(lies, remembe r)	0.4242	0.0162	0.4081	più lontane	(happy, show)	0.4278	0.0219	0.4059	più lontane	(city, old)	-0.0944	0.3048	0.3992	più vicine
coppia	anni 70	anni 2000	delta	trend																												
(tell, world)	0.4472	0.0002	0.4470	più lontane																												
(fight, two)	-0.0777	0.3315	0.4092	più vicine																												
(lies, remembe r)	0.4242	0.0162	0.4081	più lontane																												
(happy, show)	0.4278	0.0219	0.4059	più lontane																												
(city, old)	-0.0944	0.3048	0.3992	più vicine																												
270-272	<pre>for r in coppie_sist[-10:][::-1]: coppia = f"({r['w1']}, {r['w2']})" print(f"{coppia:<26} {r['s70']:>8.4f} {r['s2000']:>10.4f} {r['delta']:>7.4f}")</pre>	<code>coppie_sist[-10:]</code> prende gli ultimi 10 elementi della lista ordinata per <code>delta</code> decrescente, cioè i 10 con <code>delta</code> più basso (le coppie più stabili); <code>[::-1]</code> li inverte per mostrare prima il più stabile in assoluto (altrimenti l'ultimo elemento della lista, il minimo, verrebbe stampato per ultimo). Output reale (tutte e 10): tutte le 10 coppie più stabili mostrano <code>delta = 0.0000</code> nella stampa, es. (last, round) 0.1493 / 0.1493, (run, watch) 0.2164 / 0.2164 . Attenzione a un'illusione di precisione: questi <code>delta</code> non sono realmente zero — sono solo troppo piccoli per comparire con 4 decimali. Verificando con più cifre: per (last, round) il valore vero è <code>sim70=0.14929214</code> vs <code>sim2000=0.14928162</code>, <code>delta reale ≈ 0.0000105</code>, che arrotondato a 4 decimali diventa "0.0000". Il risultato della classifica (queste sono le coppie <i>più stabili</i> fra le 29.890, cioè quelle con <code>delta</code> minimo) resta corretto — l'ordinamento alla riga 247 usa																														

il valore `float` completo, non quello arrotondato — ma la stampa in tabella potrebbe far pensare erroneamente a un'uguaglianza esatta fra due modelli addestrati in modo completamente indipendente, cosa impossibile in pratica (i due vettori di "last" nei due modelli sono infatti completamente diversi fra loro, come ci si aspetta da due training separati).

9. Analisi sistematica delle parole comuni: deriva semantica (righe 274–346)

Approccio complementare alla sezione 8: invece di confrontare la similarità fra coppie *specifiche* di parole, qui per ogni parola frequente si confronta l'**intero vicinato** (i suoi top-N vicini più simili) fra le due epoche, misurando quanta sovrapposizione resta.

Riga	Codice	Spiegazione
276–278	<pre>FREQ_MINIMA = 30 TOP_N_VICINI = 10 TOP_RISULTATI = 30</pre>	Tre costanti di configurazione: soglia di frequenza minima per includere una parola nell'analisi (più permissiva delle 100 occorrenze della sezione 8, perché qui non si confrontano vettori a coppie ma insiemi di vicini, meno sensibili al rumore su singole parole), quanti vicini considerare per ciascuna parola, e quante parole mostrare nella classifica finale.
280–283	<pre>parole_frequenti_comuni = [p for p in sorted(comuni) if freq_70s[p] >= FREQ_MINIMA and freq_2000s[p] >= FREQ_MINIMA]</pre>	Filtra le parole comuni (insieme <code>comuni</code> della sezione 7) con almeno 30 occorrenze in entrambe le epoche, di nuovo passando per <code>sorted()</code> per un ordine deterministico (stesso motivo delle righe 212–214). Output reale: 648 parole soddisfano la soglia — quasi il triplo delle 245 usate nella sezione 8 (soglia 100), segno che abbassare la soglia di frequenza da 100 a 30 include molte più parole "moderatamente frequenti".
288–306	<pre>risultati = [] # cambiamento = 1 - overlap tra i # top-N vicini nelle due ere for parola in parole_frequenti_comuni: vicini_70s = set(w for w, _ in model_70s.wv.most_similar(parola, topn=TOP_N_VICINI)) vicini_2000s = set(w for w, _ in model_2000s.wv.most_similar(parola, topn=TOP_N_VICINI)) n_comuni_v = len(vicini_70s & vicini_2000s) overlap = n_comuni_v / TOP_N_VICINI cambiamento = 1 - overlap risultati.append({ 'parola': parola, 'cambiamento':</pre>	Per ciascuna delle 648 parole: prende i 10 vicini più simili in ciascun modello (come insiemi, per poterne calcolare l'intersezione), conta quanti vicini sono <i>in comune</i> fra le due liste (<code>n_comuni_v</code>), e ne deriva una frazione di sovrapposizione (<code>overlap</code>, fra 0 e 1) e il suo complemento (<code>cambiamento = 1 - overlap</code>, "quanto è cambiato il contesto di questa parola"). Salva anche gli insiemi di vicini completi (non solo i punteggi), perché serviranno più avanti (righe 328–338) per mostrare <i>quali</i> vicini sono spariti/comparsi.

	<pre> cambiamento, 'overlap': overlap, 'vicini_70s': vicini_70s, 'vicini_2000s': vicini_2000s, 'freq_70s': freq_70s[parola], 'freq_2000s': freq_2000s[parola], }) </pre>	
308-311	<pre> # tra le parole a pari 'cambiamento' (frequente, dato che overlap e' quantizzato # su 11 soli livelli) si preferisce mostrare prima quelle piu' frequenti: la # stima di overlap e' piu' affidabile quando il vicinato e' basato su piu' occorrenze risultati.sort(key=lambda x: (- x['cambiamento'], -(x['freq_70s'] + x['freq_2000s']), x['parola'])) </pre>	Perché il commento parla di "soli 11 livelli": con <code>TOP_N_VICINI=10</code>, <code>n_comuni_v</code> può assumere solo i valori interi 0,1,...,10 — quindi <code>overlap</code> (e di conseguenza <code>cambiamento</code>) è <i>quantizzato</i> su appena 11 valori possibili (0%, 10%, 20%, ..., 100%), il che rende le parità di punteggio non un'eccezione rara ma la norma statistica (con 648 parole e solo 11 valori possibili, per il pigeonhole principle moltissime parole condivideranno lo stesso "cambiamento"). Il criterio di ordinamento gestisce questo con tre livelli: prima per <code>cambiamento</code> decrescente, poi (a parità) per frequenza totale decrescente — motivata dal fatto che l'<code>overlap</code> calcolato su una parola con poche occorrenze è statisticamente meno affidabile di quello su una parola frequentissima — infine alfabetico come ultimo spareggio deterministico.
320-322	<pre> for i, r in enumerate(risultati[:TOP_RISULTATI]): print(f"{i+1:<4} {r['parola']}: <12} {r['cambiamento']:>11.0%} " f"{r['overlap']:>8.0%} {r['freq_70s']:>9} {r['freq_2000s']:>10}") </pre>	Stampa le 30 parole con maggior cambiamento di contesto. <code>:.0%</code> formatta un float come percentuale intera (es. <code>1.0</code> → <code>"100%"</code>). Output reale — risultato sorprendente: le prime 30 parole (fino a "around", riga 30) hanno tutte <code>cambiamento = 100%</code> e <code>overlap = 0%</code>: parole notissime come "love", "know", "like", "got", "time", "life" non condividono <i>nessuno</i> dei 10 vicini più simili fra il modello anni 70 e quello anni 2000. Perché prendere questo risultato con grande cautela (non è necessariamente "la scoperta" che sembra a prima vista): l'<code>overlap = 0%</code> per praticamente <i>tutte</i> le parole più frequenti del dataset (non solo per qualcuna) è un segnale che qualcos'altro sta succedendo oltre alla genuina deriva semantica. Le parole più frequenti come "love" o "know" compaiono in moltissimi contesti diversi all'interno di ciascun corpus; il loro vettore finale è una specie di "media" su tutti questi usi, e i suoi vicini più prossimi (top-10 su un vocabolario di migliaia di parole) tendono a essere determinati da co-occorrenze statisticamente deboli e rumorose — non dai sinonimi tematici "ovvi" che ci si aspetterebbe intuitivamente (si veda anche la nota critica alla sezione 5: molti vicini erano parole rare come "gloria" o "kandy"). Con due modelli addestrati

indipendentemente su corpora diversi, è statisticamente atteso che il rumore ad alta frequenza produca viciniati quasi completamente disgiunti anche in assenza di un vero cambiamento semantico. Questo non significa che il risultato sia "sbagliato" — la metrica fa esattamente quello che promette (misura la sovrapposizione dei viciniati) — ma va interpretato come "questo modello, con questo corpus e questi iperparametri, non produce viciniati stabili per le parole più frequenti", più che come una scoperta di forte deriva culturale. Le parole più interessanti da questo punto di vista sono probabilmente quelle nella sezione "top 10 più stabili" (righe 340–346): lì un overlap parziale ma non nullo (es. 20-30%) è un segnale semantico più robusto, perché "sopravvive" nonostante lo stesso rumore.

```
328-338 for r in risultati[:10]:
        p          = r['parola']
        solo_70s_v  = r['vicini_70s']
- r['vicini_2000s']
        solo_2000s_v =
r['vicini_2000s'] - r['vicini_70s']
        rimasti     = r['vicini_70s']
& r['vicini_2000s']

        print(f"\n[ {p.upper()} ]
cambiamento: {r['cambiamento']:.0%}
"

        f"(freq: {r['freq_70s']}
anni70 / {r['freq_2000s']}
anni2000)")
        print(f"  Vicini spariti  (solo
anni 70):  {'',
''.join(sorted(solo_70s_v))}")
        print(f"  Vicini nuovi    (solo
anni 2000): {'',
''.join(sorted(solo_2000s_v))}")
        print(f"  Rimasti stabili:
{'', ''.join(sorted(rimasti)) or
'nessuno'}")
```

Per le prime 10 parole della classifica, mostra il dettaglio esplicito di quali vicini sono usciti dal top-10, quali sono entrati, e quali sono rimasti in entrambi. `... or 'nessuno':` se `rimasti` è un insieme vuoto, `','.join(set())` produce una stringa vuota `''` — che in un contesto booleano è "falsy" — quindi l'operatore `or` sostituisce la stringa vuota (altrimenti invisibile, ambigua con "non ho stampato niente per errore") con il testo esplicito `'nessuno'`.

Output reale per "LOVE" (cambiamento 100%, freq 2669/1334):

Categoria	Parole
Vicini spariti (solo 70s)	boomaboomerang, dearly, forsake, harmony, helped, mae, mention, posses, searched, tender
Vicini nuovi (solo 2000s)	britt, elastic, everlastic, fffffuture, keizo, lovelovelove, lusting, reconciled, spastic, tragic
Rimasti stabili	nessuno

Coerentemente con la nota critica sopra, i "vicini nuovi" includono forme piuttosto insolite come "lovelovelove" (ripetizione della parola stessa, probabile refrain di una canzone) e "fffffuture" (elongazione tipica del gergo cantato/rap) — ulteriore indizio che il segnale qui è

		dominato da idiosincrasie testuali specifiche più che da un cambiamento semantico "pulito" e generalizzabile.
340-346	<pre> for r in risultati[-10:][::-1]: vicini_comuni_str = ', '.join(sorted(r['vicini_70s'] & r['vicini_2000s'])) print(f" {r['parola']:<12} overlap: {r['overlap']:.0%} " f" vicini comuni: {vicini_comuni_str}") </pre>	Le 10 parole <i>più stabili</i> (ultime della lista ordinata per cambiamento decrescente, quindi con overlap più alto), mostrate con i loro vicini effettivamente condivisi. Output reale (le 3 più stabili): "new" (overlap 30%, vicini comuni: brand, orleans, york — riconducibile all'espressione fissa "brand new" e al nome proprio "New York/New Orleans", che restano stabili semplicemente perché sono collocazioni linguistiche fisse indipendenti dall'epoca), "rest" (overlap 20%, vicini: test, vagabonds), "five" (overlap 10%, vicini: nine). Anche il caso migliore (30%) è ben lontano da un overlap alto: conferma quantitativamente quanto siano rumorosi i vicinati calcolati su questo corpus, coerentemente con la nota critica di cui sopra.

10. Clustering K-Means (righe 348–374)

Le parole più frequenti di ciascuna epoca vengono raggruppate automaticamente in 6 cluster semantici, usando l'algoritmo K-Means applicato direttamente ai vettori embedding.

Riga	Codice	Spiegazione
350–353	<pre>def fai_clustering(model, n_clusters=6, top_words=200): """Raggruppa le parole più frequenti in cluster semantici via KMeans sugli embedding.""" parole = model.wv.index_to_key[:top_words] vettori = np.array([model.wv[w] for w in parole])</pre>	<code>model.wv.index_to_key</code> è già ordinato per frequenza decrescente (nota alla sezione 7): prenderne i primi 200 (<code>top_words</code>) seleziona automaticamente le parole più frequenti senza bisogno di riordinare esplicitamente per <code>freq_70s / freq_2000s</code>. <code>vettori</code> è la matrice 200×100 dei rispettivi embedding.
354–358	<pre># KMeans usa distanza euclidea: senza normalizzare, un vettore con norma anomala # domina la distanza e finisce isolato in un cluster singoletto. Normalizzando a # norma unitaria il clustering diventa coerente con la cosine similarity usata # nel resto dell'analisi. vettori_norm = vettori / np.linalg.norm(vettori, axis=1, keepdims=True)</pre>	Motivazione tecnica cruciale: l'implementazione standard di K-Means (usata da <code>sklearn.cluster.KMeans</code>) raggruppa i punti minimizzando la distanza <i>euclidea</i>, non il coseno usato altrove nello script (<code>.similarity()</code>, <code>matrice_coseno</code>). Se due parole hanno la stessa "direzione" semantica ma norme (lunghezze) molto diverse, la distanza euclidea le considererebbe lontane anche se il coseno fra loro fosse quasi 1 — e un singolo vettore con norma anomala rischierebbe di formare un cluster isolato con un solo elemento, non per motivi semantici ma puramente numerici. Normalizzando ogni vettore a norma 1 (stessa tecnica della riga 226 in <code>matrice_coseno</code>), la distanza euclidea fra due vettori normalizzati diventa una funzione monotona del coseno fra loro, rendendo il clustering K-Means coerente con la nozione di similarità usata in tutto il resto dell'analisi.
359–364	<pre>km = KMeans(n_clusters=n_clusters, random_state=42, n_init=10) labels = km.fit_predict(vettori_norm) cluster = {} for p, l in zip(parole, labels): cluster.setdefault(l, []).append(p) return cluster, list(parole), vettori, labels</pre>	<code>random_state=42</code> fissa l'inizializzazione casuale dei centroidi per riproducibilità; <code>n_init=10</code> esegue l'intero algoritmo 10 volte con inizializzazioni diverse e tiene il risultato migliore (K-Means può convergere a un minimo locale scadente da una singola inizializzazione sfortunata). <code>fit_predict</code> restituisce, per ciascuna delle 200 parole, l'indice del cluster assegnato (0–5). Il ciclo successivo raggruppa le parole per cluster in un dizionario: <code>cluster.setdefault(l, []).append(p)</code> è l'idioma standard per "aggiungi a una lista dentro un dizionario, creandola vuota se la chiave non esiste ancora". Perché restituire anche <code>vettori</code> non normalizzati (non <code>vettori_norm</code>): il chiamante (sezione 11, t-SNE) userà i vettori originali per la proiezione dimensionale, dove la normalizzazione non è necessaria/desiderata allo stesso modo.

```

366-374 print("\nCluster anni 70 (top 200
parole, 6 cluster):")
cl_70s, par_70s, vec_70s, lbl_70s =
fai_clustering(model_70s)
for cid, parole_cl in
sorted(cl_70s.items()):
    print(f" Cluster {cid}: {'',
'.join(parole_cl[:12])}")

print("\nCluster anni 2000 (top 200
parole, 6 cluster):")
cl_2000s, par_2000s, vec_2000s,
lbl_2000s =
fai_clustering(model_2000s)
for cid, parole_cl in
sorted(cl_2000s.items()):
    print(f" Cluster {cid}: {'',
'.join(parole_cl[:12])}")

```

Esegue il clustering per entrambe le epoche e stampa, per ciascun cluster (ordinato per id), le prime 12 parole che lo compongono. Le variabili `par_70s` / `vec_70s` / `lbl_70s` (e l'equivalente per il 2000s) vengono salvate perché riusate più avanti per il grafico t-SNE (sezione 11).

Output reale, Cluster 0 di ciascuna epoca:

Anni 70 — Cluster 0: love, got, baby, yeah, want, well, take, gonna, away, let, make, right (un cluster dal tono romantico/colloquiale tipico del pop-rock)

Anni 2000 — Cluster 1: love, one, want, never, way, take, feel, right, little, every, everything, always (di nuovo un cluster a tema affettivo/romantico, ma con un id di cluster diverso: 1 invece di 0 — **i numeri di cluster non sono comparabili fra i due modelli**, dato che K-Means viene eseguito due volte in modo completamente indipendente su spazi vettoriali diversi; "Cluster 0" negli anni 70 e "Cluster 0" negli anni 2000 non hanno alcuna relazione a priori, potrebbero benissimo corrispondere a temi completamente diversi).

11. Grafico 1: proiezione t-SNE dei cluster (righe 376–436)

I 200 embedding a 100 dimensioni di ciascuna epoca vengono proiettati in un piano 2D tramite t-SNE (una tecnica di riduzione dimensionale non lineare pensata per preservare le vicinanze locali), colorati per cluster K-Means e annotati con le parole più rappresentative.

Riga	Codice	Spiegazione
378–384	<pre>def top_per_cluster(parole, labels, freq_dict, n=7): """Le n parole più frequenti di ciascun cluster, usate per etichettare i punti nel plot t-SNE.""" result = {} for cid in np.unique(labels): words_in = [p for p, l in zip(parole, labels) if l == cid] result[cid] = sorted(words_in, key=lambda w: - freq_dict.get(w, 0))[:n] return result</pre>	Perché non annotare tutte le 200 parole nel grafico: con 200 punti su un piano 2D, annotare ogni parola renderebbe il grafico illeggibile (etichette sovrapposte). Questa funzione seleziona solo le <code>n=7</code> parole più frequenti <i>all'interno di ciascun cluster</i> (non le 7 più frequenti in assoluto), garantendo che ogni cluster abbia almeno alcune etichette rappresentative anche se le sue parole non sono fra le più frequenti del vocabolario globale. <code>freq_dict.get(w, 0)</code>: usa <code>.get()</code> con default 0 invece di <code>freq_dict[w]</code> per sicurezza, nel caso (raro) una parola non fosse presente nel dizionario di frequenze passato.
386–390	<pre>fig1, axes1 = plt.subplots(1, 2, figsize=(20, 9)) fig1.suptitle('Clustering Word2Vec: Anni 70 vs Anni 2000', fontsize=16, fontweight='bold')</pre>	Griglia di 1 riga × 2 colonne (un pannello per epoca, affiancati per confronto diretto), figura larga (20 pollici) per ospitare comodamente le etichette dei 2×7=14 punti annotati per pannello.
392–397	<pre>def tsne_plot_v2(parole, vettori, labels, freq_dict, titolo, ax, n_clusters=6): """Proietta gli embedding in 2D con t-SNE, colorati per cluster e annotati solo nei punti chiave.""" n_perp = min(30, max(5, len(parole) - 1)) # perplexity limitata dal numero di punti # niente max_iter/n_iter espliciti: il nome del parametro e' cambiato tra # versioni di scikit-learn, il default (1000) e' comunque quello voluto tsne = TSNE(n_components=2, random_state=42, perplexity=n_perp)</pre>	La <code>perplexity</code> di t-SNE (un iperparametro che controlla quanti "vicini effettivi" considerare per punto, concettualmente simile a una scala di granularità) deve essere strettamente minore del numero di punti totali; <code>min(30, max(5, len(parole)-1))</code> la vincola fra 5 e 30, garantendo che sia sempre valida indipendentemente da quante parole vengono passate (qui sempre 200, quindi <code>n_perp=30</code> nella pratica, ma la funzione resta sicura anche se richiamata con meno punti). Perché non specificare <code>max_iter</code> / <code>n_iter</code> esplicitamente: il commento segnala un problema di portabilità reale — scikit-learn ha rinominato questo parametro fra versioni diverse della libreria (<code>n_iter</code> nelle versioni più vecchie, <code>max_iter</code> nelle più recenti); omettendolo del tutto e lasciando il valore di default (1000 iterazioni in entrambi i casi) evita che lo script si rompa a seconda della versione di scikit-learn installata, un compromesso pragmatico fra portabilità e controllo esplicito.

	<pre> v2d = tsne.fit_transform(vettori) </pre>	
400-404	<pre> colors = plt.cm.tab10(np.linspace(0, 0.9, n_clusters)) top_words = top_per_cluster(parole, labels, freq_dict, n=7) annotated = set() for tw_list in top_words.values(): annotated.update(tw_list) </pre>	<code>plt.cm.tab10</code> è una colormap qualitativa di matplotlib con 10 colori ben distinguibili; campionandola con <code>np.linspace(0, 0.9, 6)</code> si ottengono 6 colori equidistanti fra i primi (invece che tutti e 10, di cui solo 6 servono). <code>annotated</code> è l'unione di tutte le parole "top" di ciascun cluster (14 parole in totale, 7×2... in realtà 7 per cluster × 6 cluster = fino a 42, ma tipicamente meno per via di duplicati impossibili qui) — un <code>set</code> per un test di appartenenza rapido nel ciclo di annotazione successivo.
406-418	<pre> for cid in range(n_clusters): mask = labels == cid ax.scatter(v2d[mask, 0], v2d[mask, 1], c=[colors[cid]], label=f'C{cid}', alpha=0.5, s=50, zorder=2) for i, p in enumerate(parole): if p in annotated: ax.annotate(p, (v2d[i, 0], v2d[i, 1]), fontsize=8.5, fontweight='bold', zorder=5, bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.75, ec='none'), xytext=(4, 4), textcoords='offset points') </pre>	<code>labels == cid</code> su un array numpy produce una maschera booleana (un array di <code>True / False</code>), usata per selezionare solo i punti di quel cluster (<code>v2d[mask, 0]</code>); disegnando un cluster alla volta con colore dedicato invece di un unico scatter con array di colori, si ottiene automaticamente una legenda leggibile (<code>label=f'C{cid}'</code>). Il secondo ciclo annota solo i punti nell'insieme <code>annotated</code> (le top-7-per-cluster), con uno sfondo bianco semi-trasparente dietro il testo (<code>bbox</code>) per restare leggibile anche sopra punti colorati sovrapposti.
428-431	<pre> tsne_plot_v2(par_70s, vec_70s, lbl_70s, freq_70s, 'Anni 70 – Black Sabbath, Pink Floyd, Queen, Eagles...', axes1[0]) tsne_plot_v2(par_2000s, vec_2000s, lbl_2000s, freq_2000s, 'Anni 2000 – Eminem, Coldplay, Taylor Swift, Linkin Park...', axes1[1]) </pre>	Richiama la funzione due volte, una per pannello, riusando i risultati del clustering già calcolati alla sezione 10 (<code>par_70s</code>, <code>vec_70s</code>, <code>lbl_70s</code> e i corrispettivi 2000s) — evita di rieseguire K-Means, che è già stato fatto una volta sola.
433-436	<pre> plt.tight_layout(pad=2) plt.savefig('word2vec_cluster.png', dpi=150, bbox_inches='tight') print(" Salvato: word2vec_cluster.png") plt.close() </pre>	<code>plt.close()</code> chiude esplicitamente la figura dopo il salvataggio — perché lo script ne crea altre due (sezioni 12 e 13): senza chiuderla, matplotlib terrebbe tutte le figure aperte in memoria contemporaneamente, uno spreco di risorse inutile in uno script non interattivo dove ogni figura viene salvata su file e non serve più tenerla in memoria.

12. Grafico 2: heatmap di similarità fra parole chiave (righe 438–495)

Tre heatmap affiancate: la matrice di similarità coseno fra 12 parole chiave negli anni 70, la stessa negli anni 2000, e la loro differenza — per visualizzare in un colpo d'occhio dove le relazioni semantiche sono cambiate di più.

Riga	Codice	Spiegazione
440–442	<pre>KEYWORDS = ['love', 'life', 'night', 'heart', 'god', 'soul', 'free', 'time', 'baby', 'feel', 'world', 'dark'] kw = [p for p in KEYWORDS if p in model_70s.wv and p in model_2000s.wv]</pre>	12 parole chiave scelte manualmente (tematicamente centrali per i testi musicali), filtrate per tenere solo quelle presenti in <i>entrambi</i> i vocabolari — analogo del controllo <code>if parola in model.wv</code> visto nelle sezioni 5–6, ma qui applicato in blocco a tutta la lista invece che parola per parola con un fallback <code>'---'</code> , perché una heatmap non ha un modo sensato di rappresentare una riga/colonna "mancante".
444–451	<pre>def build_sim_matrix(model, words): """Matrice n×n delle similarità coseno a coppie tra le parole date (diagonale = 1).""" n = len(words) mat = np.zeros((n, n)) for i, w1 in enumerate(words): for j, w2 in enumerate(words): mat[i, j] = 1.0 if i == j else model.wv.similarity(w1, w2) return mat</pre>	A differenza di <code>matrice_coseno</code> (sezione 8), qui la matrice viene costruita con un doppio ciclo esplicito che chiama <code>.similarity()</code> per ogni coppia (comprese le coppie ripetute, es. sia <code>(i,j)</code> sia <code>(j,i)</code> , ridondanti perché il coseno è simmetrico) — accettabile qui perché <code>n=12</code> è piccolo (144 chiamate, contro le 29.890 coppie della sezione 8 dove la vettorizzazione era invece necessaria per le prestazioni). <code>i == j</code> forza esplicitamente la diagonale a 1.0 invece di calcolare <code>similarity(w,w)</code> : matematicamente il coseno di un vettore con sé stesso è già 1 per definizione, ma forzarlo evita qualunque imprecisione di virgola mobile (es. 0.999999998) che renderebbe la diagonale visivamente "sporca" nella heatmap.
453–456	<pre>mat70 = build_sim_matrix(model_70s, kw) mat00 = build_sim_matrix(model_2000s, kw) diff_m = mat00 - mat70 np.fill_diagonal(diff_m, 0)</pre>	Calcola le due matrici di similarità e la loro differenza elemento per elemento. <code>np.fill_diagonal(diff_m, 0)</code> : poiché entrambe le diagonali di <code>mat70 / mat00</code> sono forzate a 1.0, la loro differenza sarebbe comunque già 0 — questa riga è quindi ridondante nella pratica (una garanzia esplicita più che una correzione necessaria), ma protegge da eventuali modifiche future a <code>build_sim_matrix</code> che alterassero questa invariante.
458–474	<pre>fig2, axes2 = plt.subplots(1, 3, figsize=(22, 7)) fig2.suptitle('Similarità coseno tra parole chiave: Anni 70 vs Anni 2000', fontsize=14, fontweight='bold') for ax_i, (mat, title, cmap, vmin, vmax) in enumerate([</pre>	Itera su una lista di 3 tuple (una per pannello), ciascuna con la propria matrice, titolo, colormap e range di colore. Perché colormap e range diversi per il terzo pannello: i primi due pannelli mostrano similarità "assolute" (range naturale da -0.3 a 1.0, con <code>'RdYlGn'</code> rosso-giallo-verde dove il verde intenso = molto simili); il terzo mostra una <i>differenza</i> che può essere sia positiva sia negativa in modo simmetrico attorno allo zero, per cui usa <code>'coolwarm'</code> (blu-bianco-rosso, dove il bianco = nessun cambiamento) con range simmetrico <code>[-0.5, 0.5]</code> — usare la stessa colormap "verde=buono" per una differenza

	<pre> (mat70, 'Anni 70', 'RdYlGn', -0.3, 1.0), (mat00, 'Anni 2000', 'RdYlGn', -0.3, 1.0), (diff_m, 'Δ (2000s - 70s)', 'coolwarm', -0.5, 0.5),]): ax = axes2[ax_i] im = ax.imshow(mat, cmap=cmap, vmin=vmin, vmax=vmax, aspect='auto') ax.set_xticks(range(len(kw))) ax.set_yticks(range(len(kw))) ax.set_xticklabels(kw, rotation=45, ha='right', fontsize=9) ax.set_yticklabels(kw, fontsize=9) ax.set_title(title, fontsize=13, fontweight='bold', pad=8) plt.colorbar(im, ax=ax, shrink=0.75, pad=0.02) </pre>	non avrebbe senso semantico. <code>ax.imshow(mat, ...)</code> disegna la matrice come immagine, con ogni cella colorata secondo il proprio valore.
476-490	<pre> for i in range(len(kw)): for j in range(len(kw)): if i == j: continue val = mat[i, j] if ax_i == 2: if abs(val) < 0.05: # non annotare differenze trascurabili, per non affollare la cella continue txt = f'{val:+.2f}' color = 'white' if abs(val) > 0.3 else 'black' else: txt = f'{val:.2f}' color = 'white' if (val > 0.6 or val < -0.1) else 'black' ax.text(j, i, txt, ha='center', va='center', fontsize=7, color=color) </pre>	Annota ogni cella della heatmap con il proprio valore numerico. <code>if i == j: continue</code> salta la diagonale (già visivamente ovvia, sempre 1.0 o 0.0, non serve etichettarla). Logica diversa per il pannello differenza (<code>ax_i == 2</code>): qui i valori vicini a zero (differenza trascurabile) vengono <i>del tutto omessi</i> (<code>continue</code>) invece che stampati, per non affollare la cella con numeri poco interessanti come "+0.01"; il formato <code>{val:+.2f}</code> include sempre il segno esplicito (+ o -) per chiarire subito la direzione del cambiamento. Il colore del testo (bianco su sfondo scuro, nero su sfondo chiaro) è scelto con soglie diverse per ciascun tipo di pannello, tarate empiricamente sulle rispettive colormap per restare sempre leggibile: per i pannelli di similarità assoluta il verde/rosso più intenso si ha per <code>val > 0.6</code> o <code>val < -0.1</code> (dove serve testo bianco), per il pannello differenza per <code>abs(val) > 0.3</code>.
492-495	<pre> plt.tight_layout() plt.savefig('word2vec_heatmap.png', dpi=150, bbox_inches='tight') print(" Salvato: </pre>	Salva e chiude la seconda figura, stesso pattern della sezione 11.

```
word2vec_heatmap.png")  
plt.close()
```


13. Grafico 3: deriva semantica (righe 497–569)

Ultima figura, in due pannelli: a sinistra un grafico a barre delle 20 parole con maggior cambiamento di contesto (dati della sezione 9); a destra uno scatter plot che confronta, per migliaia di coppie di parole, la similarità negli anni 70 contro quella negli anni 2000.

Riga	Codice	Spiegazione
504–508	<pre> top_n = 20 top_d = risultati[:top_n] lbls_d = [r['parola'] for r in top_d][::-1] vals_d = [r['cambiamento'] for r in top_d][::-1] cmap_d = plt.cm.Reds(np.linspace(0.35, 0.9, top_n)) </pre>	Riusa direttamente <code>risultati</code> (già ordinato per cambiamento decrescente alla sezione 9, riga 311), prendendone le prime 20. <code>[::-1]</code> inverte l'ordine — stessa tecnica vista nel lab4 per i grafici a barre orizzontali: <code>barh</code> disegna dal basso verso l'alto, quindi invertire la lista fa apparire la parola con maggior cambiamento <i>in cima</i> al grafico.
510–522	<pre> axes3[0].barh(range(top_n), vals_d, color=cmap_d, alpha=0.9, height=0.7) axes3[0].set_yticks(range(top_n)) axes3[0].set_yticklabels(lbls_d, fontsize=10) axes3[0].set_xlabel('Grado di cambiamento del contesto semantico', fontsize=11) axes3[0].set_title(f'Top {top_n} parole con maggiore deriva semantica', fontsize=12, fontweight='bold') axes3[0].set_xlim(0, 1.18) for i, v in enumerate(vals_d): axes3[0].text(v + 0.02, i, f'{v:.0%}', va='center', fontsize=9) axes3[0].axvline(x=0.5, color='gray', linestyle='--', alpha=0.4, label='soglia 50%') axes3[0].legend(fontsize=9) </pre>	Grafico a barre orizzontali con etichette percentuali a fianco di ciascuna barra. <code>set_xlim(0, 1.18)</code>: dato che <code>cambiamento</code> arriva al massimo a 1.0 (100%), il margine extra fino a 1.18 lascia spazio per le etichette testuali (<code>v + 0.02</code>) senza che vengano tagliate dal bordo del grafico. <code>axvline</code> disegna una linea tratteggiata verticale a 0.5 come riferimento visivo di "metà cambiamento" — utile qui perché, come emerso dalla sezione 9, moltissime barre saranno vicine o pari al 100%, quindi la soglia aiuta a distinguere visivamente le parole "estreme" dal resto.
525–532	<pre> ax_s = axes3[1] n_plot = min(800, len(coppie_sist)) s70_v = [r['s70'] for r in coppie_sist[:n_plot]] s00_v = [r['s2000'] for r in coppie_sist[:n_plot]] </pre>	Perché limitare a <code>min(800, len(coppie_sist))</code> invece di disegnare tutte le 29.890 coppie: uno scatter plot con quasi 30.000 punti sarebbe visivamente un'unica macchia indistinguibile (overplotting) e appesantirebbe inutilmente il file immagine; <code>coppie_sist</code> è già ordinato per <code>delta</code> decrescente (sezione 8, riga 247), quindi prendendo i primi

	<pre> d_v = [r['delta'] for r in coppie_sist[:n_plot]] sc = ax_s.scatter(s70_v, s00_v, c=d_v, cmap='YlOrRd', alpha=0.5, s=18, vmin=0, vmax=0.35, zorder=2) </pre>	800 si mostrano le coppie <i>più interessanti</i> (quelle con maggior cambiamento), non un campione casuale. Ogni punto è colorato in base al proprio <code>delta</code> (<code>c=d_v</code>), rendendo il colore ridondante ma rinforzativo rispetto alla posizione (i punti più lontani dalla diagonale, che si vedrà definita subito sotto, sono anche quelli con colore più intenso).
534-539	<pre> all_vals = s70_v + s00_v lo = min(all_vals) - 0.05 hi = max(all_vals) + 0.05 ax_s.plot([lo, hi], [lo, hi], 'k--', alpha=0.4, lw=1.5, label='nessun cambiamento') ax_s.fill_between([lo, hi], [lo, hi], [hi, hi], alpha=0.04, color='steelblue') ax_s.fill_between([lo, hi], [lo, lo], [lo, hi], alpha=0.04, color='tomato') </pre>	<code>all_vals = s70_v + s00_v</code>: qui <code>+</code> fra due liste Python è <i>concatenazione</i>, non somma elemento-per-elemento (non è aritmetica su array numpy) — il risultato è una lista con tutti i 1600 valori (800 <code>s70</code> + 800 <code>s2000</code>) mescolati insieme, usata solo per trovarne il minimo/massimo globale e calcolare un range <code>[lo, hi]</code> comune per entrambi gli assi. La linea tratteggiata <code>y=x</code> rappresenta "nessun cambiamento" (dove <code>s70 == s2000</code>): i punti sopra questa diagonale sono coppie diventate più simili nei 2000, quelli sotto sono diventate meno simili. I due <code>fill_between</code> colorano leggermente (<code>alpha=0.04</code>, quasi trasparente) le due metà del grafico sopra/sotto la diagonale, come sfondo di orientamento visivo.
541-546	<pre> # coordinate relative agli assi (0- 1), non ai dati: restano nell'angolo giusto # qualunque sia il range effettivo di lo/hi ax_s.text(0.03, 0.95, 'più vicine\nei 2000s', transform=ax_s.transAxes, fontsize=8, color='steelblue', alpha=0.8, va='top') ax_s.text(0.75, 0.05, 'più lontane\nei 2000s', transform=ax_s.transAxes, fontsize=8, color='tomato', alpha=0.8, va='bottom') </pre>	Perché <code>transform=ax_s.transAxes</code>: senza questo parametro, le coordinate <code>(0.03, 0.95)</code> passate a <code>ax.text()</code> verrebbero interpretate come coordinate <i>nei dati</i> (cioè valori di similarità 0.03 e 0.95), che potrebbero cadere fuori dal grafico o in un punto sbagliato a seconda del range effettivo <code>[lo, hi]</code> calcolato dinamicamente alla riga 535-536. <code>transAxes</code> reinterpreta le stesse coordinate come frazioni dell'area del grafico (0,0 = angolo in basso a sinistra, 1,1 = angolo in alto a destra), indipendentemente dai valori numerici degli assi — così l'etichetta "più vicine nei 2000s" resta sempre nell'angolo in alto a sinistra qualunque sia il range dei dati.
548-552	<pre> for r in coppie_sist[:8]: ax_s.annotate(f"{r['w1']}- {r['w2']}", (r['s70'], r['s2000']), fontsize=7.5, alpha=0.85, xytext=(6, 4), textcoords='offset points', bbox=dict(boxstyle='round,pad=0.1', fc='white', alpha=0.6, ec='none')) </pre>	Annota testualmente solo le 8 coppie con il delta più alto in assoluto (di nuovo grazie all'ordinamento già fatto), per dare un riferimento concreto ai punti più estremi dello scatter — coerente con la tabella "TOP 20" già stampata testualmente alla sezione 8, di cui questo grafico è la controparte visiva.
554-564	<pre> ax_s.set_xlim(lo, hi) ax_s.set_ylim(lo, hi) ax_s.set_xlabel('Similarità coseno – Anni 70', fontsize=11) ax_s.set_ylabel('Similarità coseno </pre>	<code>set_xlim / set_ylim</code> allo stesso range <code>[lo,hi]</code> per entrambi gli assi è essenziale: senza aspetto quadrato/range coincidenti, la diagonale <code>y=x</code> disegnata alla riga 537 non apparirebbe visivamente a 45 gradi, rendendo l'interpretazione "sopra/sotto la diagonale" fuorviante.

	<pre> – Anni 2000', fontsize=11) ax_s.set_title('Coppie di parole: 70s vs 2000s\n(sopra diagonale = più simili nei 2000s)', fontsize=12, fontweight='bold') plt.colorbar(sc, ax=ax_s, label=' Δ similarità ', shrink=0.8) ax_s.legend(fontsize=9, loc='upper left') ax_s.grid(True, alpha=0.2) </pre>	
566-569	<pre> plt.tight_layout(pad=2) plt.savefig('word2vec_drift.png', dpi=150, bbox_inches='tight') print(" Salvato: word2vec_drift.png") plt.close() </pre>	Salva e chiude la terza e ultima figura.

14. Riepilogo finale (righe 571–603)

Ultimo blocco: ripete alcune interrogazioni chiave (parole più vicine a "love", "night", "feel") già viste nella sezione 5, e un riepilogo numerico finale del vocabolario — un "executive summary" leggibile senza scorrere tutto l'output precedente.

Riga	Codice	Spiegazione
573–577	<pre>print("\n=== PAROLE PIU' VICINE A 'love' ===") if 'love' in model_70s.wv: print(f"Anni 70: {[w for w,_ in model_70s.wv.most_similar('love', topn=8)]}") if 'love' in model_2000s.wv: print(f"Anni 2000: {[w for w,_ in model_2000s.wv.most_similar('love', topn=8)]}")</pre>	Stesso pattern della sezione 5 (<code>most_similar</code> con controllo di appartenenza al vocabolario), ma qui con <code>topn=8</code> invece di 3, e con due <code>if</code> separati invece dell'espressione condizionale inline — uno stile leggermente diverso per lo stesso controllo, segno che questo blocco finale è stato probabilmente scritto in un momento diverso rispetto alla sezione 5, senza fattorizzare la logica comune in una funzione condivisa. Output reale: Anni 70 → forsake, harmony, tender, dearly, posses, mention, helped, searched; Anni 2000 → everlastic, lusting, elastic, spastic, lovelovelove, fffffuture, reconciled, tragic. Coerente (le prime 3 di ciascuna lista coincidono) con l'output della sezione 5, che usava le stesse chiamate con <code>topn=3</code> : qui si vedono semplicemente altre 5 parole in più della stessa classifica di similarità.
579–589	<pre>print("\n=== PAROLE PIU' VICINE A 'night' ===") ... (stesso pattern per 'night' e poi per 'feel')</pre>	Ripete lo stesso blocco per altre due parole chiave ("night", "feel"), duplicando il codice invece di iterare su una lista come fatto altrove (es. <code>parole_test</code> alla sezione 5) — probabilmente perché qui si vogliono solo 3 parole specifiche per il riepilogo finale, non l'intero set di 7 della sezione 5.
591–596	<pre>print(f"\n=== VOCABOLARIO ===") print(f"Anni 70: {len(model_70s.wv)} parole") print(f"Anni 2000: {len(model_2000s.wv)} parole") print(f"Solo anni 70: {len(solo_70s)}") print(f"Solo anni 2000: {len(solo_2000s)}") print(f"In comune: {len(comuni)}")</pre>	Riepiloga in poche righe le statistiche di vocabolario già stampate nelle sezioni 4 e 7, per comodità di lettura senza dover risalire tutto l'output.
598–602	<pre>print("\nLab completato!") print("File generati:") print(" - word2vec_cluster.png (t-SNE clustering 70s vs 2000s)") print(" - word2vec_heatmap.png</pre>	Messaggio finale con l'elenco esplicito dei 3 file immagine prodotti durante l'esecuzione (sezioni 11–13) — utile perché, a differenza dei <code>print()</code> testuali che scorrono nel terminale, i file PNG restano sul disco e serve un promemoria di dove trovarli.

```
(heatmap similarità parole  
chiave")  
print(" - word2vec_drift.png  
(deriva semantica + scatter  
coppie")
```

Glossario dei concetti chiave

Riepilogo dei concetti di NLP/machine learning usati nel codice, per chi vuole un ripasso rapido senza tornare al materiale del corso.

Word2Vec

Modello che apprende una rappresentazione vettoriale densa (embedding) per ogni parola di un vocabolario, tale che parole usate in contesti simili ottengano vettori vicini nello spazio. Addestrato qui con la variante **skip-gram** (`sg=1`): dato una parola centrale, il modello impara a predire le parole del suo contesto (finestra di `window` parole prima e dopo).

Embedding

Il vettore di numeri reali (qui, 100 dimensioni) associato a ciascuna parola dal modello addestrato. Non ha dimensioni interpretabili singolarmente: il significato emerge solo dalle relazioni geometriche fra vettori (distanza, coseno) rispetto ad altri embedding.

Similarità coseno

Misura di quanto due vettori "puntano nella stessa direzione", indipendentemente dalla loro lunghezza: $\cos(a,b) = (a \cdot b) / (\|a\| \|b\|)$, compresa fra -1 (opposti) e 1 (identica direzione). È la metrica standard per confrontare embedding di parole, perché la lunghezza del vettore spesso riflette solo la frequenza della parola, non il suo significato.

min_count, window, epochs (iperparametri Word2Vec)

`min_count`: frequenza minima per entrare nel vocabolario. `window`: quante parole di contesto considerare attorno a ciascuna parola centrale. `epochs`: quante volte il corpus viene attraversato durante il training — più epoche aiutano su corpora piccoli, a rischio di overfitting su corpora troppo ripetitivi.

K-Means

Algoritmo di clustering che partiziona un insieme di punti (qui, embedding di parole) in k gruppi, minimizzando la distanza euclidea di ciascun punto dal centroide del proprio gruppo. Richiede di specificare k in anticipo (qui, 6) e dipende dall'inizializzazione casuale dei centroidi (mitigata da `n_init=10`).

t-SNE

Tecnica di riduzione dimensionale non lineare, usata qui per proiettare embedding a 100 dimensioni in un piano 2D visualizzabile. A differenza di PCA (lineare), t-SNE cerca di preservare le vicinanze *locali* fra punti, a costo di distorcere le distanze globali — utile per visualizzare cluster, non per misurare distanze assolute sul grafico.

Perplexity (di t-SNE)

Da non confondere con la perplexity dei modelli linguistici (lab4): qui è un iperparametro di t-SNE che controlla approssimativamente quanti vicini locali ogni punto considera nella proiezione — valori tipici fra 5 e 50, deve essere minore del numero di punti totali.

Deriva semantica (semantic drift)

Il fenomeno per cui il significato/uso di una parola cambia nel tempo o fra contesti diversi. In questo script viene operazionalizzata in due modi: (1) variazione della similarità coseno fra coppie specifiche di parole (sezione 8); (2) sovrapposizione dei vicini più simili di una parola (sezione 9) — entrambe misure indirette, sensibili al rumore su corpora piccoli come discusso nelle note critiche del documento.